

```

import { useState, useEffect } from "react";

const TOKENS = ["Time", "flies", "like", "an", "arrow"];
const COLORS = {
  bg: "#0a0e17",
  surface: "#131825",
  surfaceHover: "#1a2035",
  border: "#1e2a42",
  accent: "#4f8ff7",
  accentDim: "#2a4a8a",
  key: "#f59e0b",
  keyDim: "#92610d",
  value: "#10b981",
  valueDim: "#0a7a56",
  query: "#a78bfa",
  queryDim: "#6b5aad",
  text: "#e2e8f0",
  textMuted: "#64748b",
  danger: "#ef4444",
  dangerDim: "#991b1b",
};

const font = `JetBrains Mono, Fira Code, SF Mono, monospace`;
const sansFont = `DM Sans, Segoe UI, system-ui, sans-serif`;

export default function KVCacheExplainer() {
  const [step, setStep] = useState(0);
  const [animating, setAnimating] = useState(false);

  const steps = [
    {
      title: "The Problem: Redundant Computation",
      desc: "Without a KV cache, the model recomputes Keys and Values for ALL pre",
      mode: "no-cache",
    },
    {
      title: "What Are Q, K, V?",
      desc: "Each token is projected into three vectors through learned weight ma",
      mode: "qkv",
    },
    {
      title: "KV Cache: Store & Reuse",
      desc: "The insight: Keys and Values for past tokens don't change. Cache the",
      mode: "cache",
    },
  ];

```

```

    },
    {
      title: "Memory Growth Problem",
      desc: "The cache grows linearly with sequence length. For each layer and ea
      mode: "memory",
    },
    {
      title: "Where TurboQuant Fits",
      desc: "TurboQuant compresses the cached K/V vectors from FP16 (~16 bits) do
      mode: "turboquant",
    },
  ],
];

```

```

const goTo = (s) => {
  if (animating) return;
  setAnimating(true);
  setStep(s);
  setTimeout(() => setAnimating(false), 400);
};

```

```

return (
  <div style={{
    background: COLORS.bg,
    minHeight: "100vh",
    color: COLORS.text,
    fontFamily: sansFont,
    padding: "24px 16px",
    boxSizing: "border-box",
  }}>
    <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;7
    <div style={{ maxWidth: 720, margin: "0 auto" }}>
      <div style={{ marginBottom: 28, textAlign: "center" }}>
        <h1 style={{
          fontFamily: font,
          fontSize: 22,
          fontWeight: 700,
          margin: 0,
          letterSpacing: "-0.5px",
          background: `linear-gradient(135deg, ${COLORS.key}, ${COLORS.value})`
          WebkitBackgroundClip: "text",
          WebkitTextFillColor: "transparent",
        }}>
          KV Cache
        </h1>
        <p style={{ color: COLORS.textMuted, fontSize: 13, margin: "4px 0 0" }}

```

How Transformers Remember During Generation

</p>

</div>

```
{/* Step nav */}
```

```
<div style={{
```

```
  display: "flex",
```

```
  gap: 4,
```

```
  marginBottom: 20,
```

```
  justifyContent: "center",
```

```
}}>
```

```
  {steps.map((s, i) => (
```

```
    <button
```

```
      key={i}
```

```
      onClick={() => goTo(i)}
```

```
      style={{
```

```
        flex: 1,
```

```
        maxWidth: 120,
```

```
        height: 4,
```

```
        borderRadius: 2,
```

```
        border: "none",
```

```
        background: i <= step ? COLORS.accent : COLORS.border,
```

```
        cursor: "pointer",
```

```
        transition: "background 0.3s",
```

```
      }})
```

```
    />
```

```
  )})
```

```
</div>
```

```
{/* Content card */}
```

```
<div style={{
```

```
  background: COLORS.surface,
```

```
  border: `1px solid ${COLORS.border}`,
```

```
  borderRadius: 14,
```

```
  padding: 20,
```

```
  marginBottom: 16,
```

```
  transition: "opacity 0.3s",
```

```
  opacity: animating ? 0.7 : 1,
```

```
}}>
```

```
  <div style={{
```

```
    fontSize: 11,
```

```
    color: COLORS.accent,
```

```
    fontFamily: font,
```

```
    fontWeight: 500,
```

```
    marginBottom: 6,
```

```
    textTransform: "uppercase",
```

```
    letterSpacing: "1.5px",
```

```

    >>
    Step {step + 1} / {steps.length}
  </div>
  <h2 style={{
    fontSize: 18,
    fontWeight: 700,
    margin: "0 0 8px",
    lineHeight: 1.3,
  }}>
    {steps[step].title}
  </h2>
  <p style={{
    fontSize: 13,
    lineHeight: 1.65,
    color: COLORS.textMuted,
    margin: 0,
  }}>
    {steps[step].desc}
  </p>
</div>

{/* Visualization area */}
<div style={{
  background: COLORS.surface,
  border: `1px solid ${COLORS.border}`,
  borderRadius: 14,
  padding: 20,
  minHeight: 300,
}}>
  {step === 0 && <NoCacheViz />}
  {step === 1 && <QKV Viz />}
  {step === 2 && <CacheViz />}
  {step === 3 && <MemoryViz />}
  {step === 4 && <TurboQuantViz />}
</div>

{/* Navigation */}
<div style={{
  display: "flex",
  gap: 10,
  marginTop: 16,
  justifyContent: "space-between",
}}>
  <button
    onClick={() => step > 0 && goTo(step - 1)}
    disabled={step === 0}
    style={{

```

```

        ...btnStyle,
        opacity: step === 0 ? 0.3 : 1,
        cursor: step === 0 ? "default" : "pointer",
      }}
    >
      ← Back
    </button>
    <button
      onClick={() => step < steps.length - 1 && goTo(step + 1)}
      disabled={step === steps.length - 1}
      style={{
        ...btnStyle,
        background: step === steps.length - 1 ? COLORS.border : COLORS.acce
        color: "#fff",
        cursor: step === steps.length - 1 ? "default" : "pointer",
        opacity: step === steps.length - 1 ? 0.5 : 1,
      }}
    >
      Next →
    </button>
  </div>
</div>
</div>
);
}

```

```

const btnStyle = {
  fontFamily: `'DM Sans', system-ui, sans-serif`,
  fontSize: 13,
  fontWeight: 600,
  padding: "10px 24px",
  borderRadius: 8,
  border: `1px solid ${COLORS.border}`,
  background: COLORS.surfaceHover,
  color: COLORS.text,
  transition: "all 0.2s",
};

```

/* — Step 0: No Cache — */

```

function NoCacheViz() {
  const [genStep, setGenStep] = useState(0);
  useEffect(() => {
    const t = setInterval(() => setGenStep((s) => (s + 1) % 5), 2000);
    return () => clearInterval(t);
  }, []);

  const currentLen = genStep + 1;

```

```

const totalOps = Array.from({ length: currentLen }, (_, i) => i + 1).reduce((a,

return (
  <div>
    <div style={{ fontSize: 12, color: COLORS.textMuted, marginBottom: 14, font
      Generating token {currentLen} of {TOKENS.length} – recomputing {totalOps}
    </div>
    <div style={{ display: "flex", flexDirection: "column", gap: 8 }}>
      {TOKENS.slice(0, currentLen).map((tok, i) => (
        <div key={i} style={{ display: "flex", alignItems: "center", gap: 8 }}>
          <TokenChip token={tok} active={i === currentLen - 1} />
          <div style={{ display: "flex", gap: 4 }}>
            <MiniVec label="K" color={COLORS.key} dim={i < currentLen - 1} />
            <MiniVec label="V" color={COLORS.value} dim={i < currentLen - 1} />
          </div>
          {i < currentLen - 1 && (
            <span style={{ fontSize: 10, color: COLORS.danger, fontFamily: font
              RECOMPUTED
            </span>
          )}
          {i === currentLen - 1 && (
            <span style={{ fontSize: 10, color: COLORS.accent, fontFamily: font
              NEW
            </span>
          )}
        </div>
      )}}
    </div>
    <div style={{
      marginTop: 16,
      padding: 12,
      borderRadius: 8,
      background: COLORS.dangerDim + "33",
      border: `1px solid ${COLORS.danger}33`,
      fontSize: 12,
      color: COLORS.danger,
      fontFamily: font,
    }}>
      Wasted compute: {totalOps - 1} redundant K,V projections at step {current
    </div>
  </div>
);
}

/* — Step 1: Q K V Explainer — */
function QKVViz() {
  return (

```

```

<div>
  <div style={{ display: "flex", flexDirection: "column", gap: 14 }}>
    <VectorRow
      label="Query (Q)"
      subtitle={`"What am I looking for?"`}
      color={COLORS.query}
      emoji="🔍"
    />
    <VectorRow
      label="Key (K)"
      subtitle={`"What do I contain?"`}
      color={COLORS.key}
      emoji="🔑"
    />
    <VectorRow
      label="Value (V)"
      subtitle={`"Here's my information"`}
      color={COLORS.value}
      emoji="📦"
    />
  </div>

  <div style={{
    marginTop: 20,
    padding: 14,
    borderRadius: 10,
    background: COLORS.surfaceHover,
    border: `1px solid ${COLORS.border}`,
  }}>
    <div style={{ fontFamily: font, fontSize: 13, color: COLORS.accent, margin: 0 0 10px 0 }}>
      Attention Mechanism
    </div>
    <div style={{ fontFamily: font, fontSize: 14, color: COLORS.text, lineHeight: 1.2 }}>
      <span style={{ color: COLORS.query }}>Q</span>
      {" · "}
      <span style={{ color: COLORS.key }}>KT</span>
      {" → score how relevant each past token is"}
    </div>
    <div style={{ fontFamily: font, fontSize: 14, color: COLORS.text, lineHeight: 1.2 }}>
      {"softmax(scores / √d) → normalize to weights"}
    </div>
    <div style={{ fontFamily: font, fontSize: 14, color: COLORS.text, lineHeight: 1.2 }}>
      {"weights × "}
      <span style={{ color: COLORS.value }}>V</span>
      {" → weighted sum of information"}
    </div>
  </div>
</div>

```

```

    <div style={{
      marginTop: 12,
      fontSize: 12,
      color: COLORS.textMuted,
      lineHeight: 1.6,
    }}>
      Key insight: <strong style={{ color: COLORS.text }}>Q changes every step<
    </div>
  </div>
);
}

/* — Step 2: Cache — */
function CacheViz() {
  const [genStep, setGenStep] = useState(0);
  useEffect(() => {
    const t = setInterval(() => setGenStep((s) => (s + 1) % 5), 2200);
    return () => clearInterval(t);
  }, []);

  const currentLen = genStep + 1;

  return (
    <div>
      <div style={{ fontSize: 12, color: COLORS.textMuted, marginBottom: 14, font
        Generating token {currentLen} — only computing K,V for the new token
      </div>

      {/* Cache */}
      <div style={{
        padding: 12,
        borderRadius: 10,
        border: `1px dashed ${COLORS.accentDim}`,
        background: COLORS.accentDim + "15",
        marginBottom: 12,
      }}>
        <div style={{ fontSize: 10, color: COLORS.accent, fontFamily: font, fontW
          KV Cache (stored in VRAM)
        </div>
        <div style={{ display: "flex", flexDirection: "column", gap: 6 }}>
          {TOKENS.slice(0, currentLen).map((tok, i) => (
            <div key={i} style={{
              display: "flex",
              alignItems: "center",
              gap: 8,
              opacity: i === currentLen - 1 ? 1 : 0.7,

```

```

        transition: "opacity 0.3s",
    }}>
    <TokenChip token={tok} active={i === currentLen - 1} />
    <MiniVec label="K" color={COLORS.key} dim={false} />
    <MiniVec label="V" color={COLORS.value} dim={false} />
    {i === currentLen - 1 ? (
        <span style={{ fontSize: 10, color: COLORS.value, fontFamily: font
            ← APPEND
        </span>
    ) : (
        <span style={{ fontSize: 10, color: COLORS.textMuted, fontFamily:
            cached
        </span>
    )}
    </div>
    )}}
</div>
</div>

<div style={{
    padding: 12,
    borderRadius: 8,
    background: COLORS.valueDim + "33",
    border: `1px solid ${COLORS.value}33`,
    fontSize: 12,
    color: COLORS.value,
    fontFamily: font,
}}>
    Only 1 new K,V computed per step → O(1) per token instead of O(n)
</div>
</div>
);
}

```

/* — Step 3: Memory Growth — */

```

function MemoryViz() {
    const contexts = [1, 2, 4, 8, 16, 32, 64, 128];

    return (
        <div>
            <div style={{ fontSize: 12, color: COLORS.textMuted, marginBottom: 14, font
                KV cache memory for a ~7B model (32 layers, 32 heads, d=128, FP16)
            </div>
            <div style={{ display: "flex", flexDirection: "column", gap: 6 }}>
                {contexts.map((ctx) => {
                    const memGB = (ctx * 1024 * 32 * 32 * 128 * 2 * 2) / (1024 ** 3);
                    const maxWidth = 100;

```

```

const barWidth = Math.min((memGB / 32) * maxWidth, maxWidth);
const danger = memGB > 16;
return (
  <div key={ctx} style={{ display: "flex", alignItems: "center", gap: 8
    <div style={{ width: 45, textAlign: "right", fontFamily: font, font
      {ctx}K
    </div>
    <div style={{
      flex: 1,
      height: 20,
      background: COLORS.surfaceHover,
      borderRadius: 4,
      overflow: "hidden",
      position: "relative",
    }}>
      <div style={{
        width: `${barWidth}%`,
        height: "100%",
        background: danger
          ? `linear-gradient(90deg, ${COLORS.key}, ${COLORS.danger})`
          : `linear-gradient(90deg, ${COLORS.accentDim}, ${COLORS.accentDim}`
        borderRadius: 4,
        transition: "width 0.5s ease",
      }} />
    </div>
    <div style={{
      width: 55,
      fontFamily: font,
      fontSize: 11,
      color: danger ? COLORS.danger : COLORS.textMuted,
      fontWeight: danger ? 700 : 400,
    }}>
      {memGB.toFixed(1)} GB
    </div>
  </div>
);
)}}
</div>
<div style={{
  marginTop: 16,
  padding: 12,
  borderRadius: 8,
  background: COLORS.dangerDim + "33",
  border: `1px solid ${COLORS.danger}33`,
  fontSize: 12,
  color: COLORS.text,
  lineHeight: 1.6,

```

```

    }}>
    <strong style={{ color: COLORS.danger }}>At 128K context</strong>, the KV
  </div>
</div>
);
}

```

```

/* — Step 4: TurboQuant — */

```

```

function TurboQuantViz() {
  const tokens = ["The", "cat", "sat", "on", "the", "mat", "and", "then"];
  const recentWindow = 2;

  return (
    <div>
      <div style={{ fontSize: 12, color: COLORS.textMuted, marginBottom: 14, font
        TurboQuant compresses older cache entries, keeps recent tokens at full pr
      </div>
      <div style={{ display: "flex", flexDirection: "column", gap: 5 }}>
        {tokens.map((tok, i) => {
          const isRecent = i >= tokens.length - recentWindow;
          return (
            <div key={i} style={{
              display: "flex",
              alignItems: "center",
              gap: 8,
              padding: "6px 10px",
              borderRadius: 8,
              background: isRecent ? COLORS.accentDim + "25" : COLORS.surfaceHove
              border: `1px solid ${isRecent ? COLORS.accent + "44" : "transparent
            }}>
            <div style={{
              fontFamily: font,
              fontSize: 12,
              color: isRecent ? COLORS.text : COLORS.textMuted,
              width: 36,
            }}>
              {tok}
            </div>

          </div>

          {/K vector visualization */}
          <div style={{ display: "flex", gap: 2 }}>
            {Array.from({ length: 8 }).map((_, j) => (
              <div key={j} style={{
                width: isRecent ? 16 : 6,
                height: 12,
                borderRadius: 2,
                background: isRecent ? COLORS.key : COLORS.keyDim,

```

```

        opacity: isRecent ? 0.9 : 0.5,
        transition: "all 0.3s",
      }} />
    )))
  </div>

  <div style={{
    fontFamily: font,
    fontSize: 9,
    color: isRecent ? COLORS.text : COLORS.textMuted,
    fontWeight: 600,
    whiteSpace: "nowrap",
  }}>
    {isRecent ? "FP16 (16-bit)" : "3-bit ⚡"}
  </div>

  {!isRecent && (
    <div style={{
      fontSize: 9,
      color: COLORS.value,
      fontFamily: font,
      marginLeft: "auto",
    }}>
      ~5× smaller
    </div>
  )}
</div>

);
)))
</div>

<div style={{
  marginTop: 14,
  display: "flex",
  gap: 8,
}}>
  <CompactCard
    title="PolarQuant"
    desc="Rotate vectors → predictable distribution → optimal quantization"
    color={COLORS.key}
    step="1"
  />
  <CompactCard
    title="QJL"
    desc="1-bit residual error correction → unbiased dot products"
    color={COLORS.value}
    step="2"
  />

```

```

    />
  </div>

  <div style={{
    marginTop: 12,
    padding: 12,
    borderRadius: 8,
    background: COLORS.valueDim + "33",
    border: `1px solid ${COLORS.value}33`,
    fontSize: 12,
    color: COLORS.value,
    fontFamily: font,
    lineHeight: 1.5,
  }}>
    No training, no calibration, model-agnostic. Same codebook works for any
  </div>
</div>
);
}

/* — Shared Components — */
function TokenChip({ token, active }) {
  return (
    <div style={{
      fontFamily: font,
      fontSize: 12,
      fontWeight: active ? 700 : 500,
      padding: "3px 10px",
      borderRadius: 6,
      background: active ? COLORS.accent + "30" : COLORS.surfaceHover,
      border: `1px solid ${active ? COLORS.accent : COLORS.border}`,
      color: active ? COLORS.accent : COLORS.textMuted,
      minWidth: 45,
      textAlign: "center",
    }}>
      {token}
    </div>
  );
}

function MiniVec({ label, color, dim }) {
  return (
    <div style={{
      display: "flex",
      alignItems: "center",
      gap: 3,
      padding: "2px 6px",

```

```

borderRadius: 4,
background: dim ? "transparent" : color + "20",
border: `1px solid ${dim ? color + "33" : color + "55"}`,
}}>
<span style={{ fontFamily: font, fontSize: 10, fontWeight: 700, color: dim
  {label}
</span>
<div style={{ display: "flex", gap: 1 }}>
  [[0.6, 0.9, 0.3, 0.7].map((h, i) => (
    <div key={i} style={{
      width: 3,
      height: 10 * h + 4,
      borderRadius: 1,
      background: dim ? color + "44" : color + "88",
    }} />
  ))]
</div>
</div>
);
}

```

```

function VectorRow({ label, subtitle, color, emoji }) {
  return (
    <div style={{
      display: "flex",
      alignItems: "center",
      gap: 12,
      padding: 12,
      borderRadius: 10,
      background: color + "12",
      border: `1px solid ${color}33`,
    }}>
      <div style={{ fontSize: 22 }}>{emoji}</div>
      <div>
        <div style={{ fontFamily: font, fontSize: 13, fontWeight: 700, color }}>{
          <div style={{ fontSize: 12, color: COLORS.textMuted, marginTop: 2 }}>{sub
        </div>
        <div style={{ marginLeft: "auto", display: "flex", gap: 2 }}>
          [Array.from({ length: 6 }).map((_, i) => (
            <div key={i} style={{
              width: 4,
              height: 8 + Math.random() * 14,
              borderRadius: 2,
              background: color + "66",
            }} />
          ))]
        </div>
      </div>
    </div>
  );
}

```

```

    </div>
  );
}

function CompactCard({ title, desc, color, step }) {
  return (
    <div style={{
      flex: 1,
      padding: 10,
      borderRadius: 8,
      background: COLORS.surfaceHover,
      border: `1px solid ${COLORS.border}`,
    }}>
      <div style={{ display: "flex", alignItems: "center", gap: 6, marginBottom:
        <div style={{
          width: 18,
          height: 18,
          borderRadius: "50%",
          background: color + "33",
          display: "flex",
          alignItems: "center",
          justifyContent: "center",
          fontFamily: font,
          fontSize: 10,
          fontWeight: 700,
          color,
        }}>
          {step}
        </div>
        <div style={{ fontFamily: font, fontSize: 11, fontWeight: 700, color }}>{
      </div>
      <div style={{ fontSize: 10, color: COLORS.textMuted, lineHeight: 1.5 }}>{de
    </div>
  );
}

```